

Project Report

Details:

Name: Adarsh V Pai

Role: Electronics and PCB Designing Intern

Introduction for Cube Mini Prototype

On 3rd March we had a discussion about cube mini prototype and it's working. Saw a demo of how the Cube ai works and it's speech to text capabilities

We were given a new task on Sound Source Localization.

- Learnt about Sipeed MAIX-I module and its working.
- Learnt about I2S communication Protocol.
- Looked into the WebRTS introduction and Gear Motor working.

Peer to Peer connection between Raspberry Pi and Laptop

On 4th March, a task was given to implement a P2P connection between Raspberry Pi and Laptop and sent an audio file from pi to laptop

- Looked into WebRTS and tried to implement the connection using WebRTC.
- Tried so many ways but it was very difficult to implement the task using WebRTC.
- On 5th March, tried the same using Websockets and successfully transferred an audio file.

Receiver Code(Server):

```
import asyncio
import websockets
async def receive_audio(websocket):
    print("Connection established. Receiving audio file...")
    with open("received_audio.mp3", "wb") as audio_file: # Save as MP3
        try:
            async for data in websocket:
                if isinstance(data, bytes): # Ensure binary data
                    audio_file.write(data)
                else:
                    print("Received non-binary data:", data)
        except websockets.exceptions.ConnectionClosed:
            print("Connection closed.")
    print("Audio file received successfully!")
async def main():
    async with websockets.serve(receive_audio, "0.0.0.0", 8765):
        await asyncio.Future() # Keep the server running
asyncio.run(main())
```

The receiver code acts as a WebSocket server that listens for incoming connections on port **8765** using Python's websockets library. It uses the asyncio library to handle multiple connections asynchronously. The function `receive_audio(websocket)` is triggered when a client connects. It opens a file named

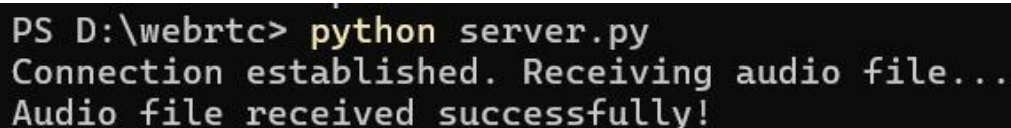
received_audio.mp3 in **binary write mode** (wb) and starts receiving data from the WebSocket connection. The `async` for data in `websocket: loop` ensures that data is received continuously. If the received data is in **binary format (bytes)**, it is written to the file; otherwise, non-binary data is printed for debugging. If the client disconnects, `websockets.exceptions.ConnectionClosed` is caught, and the server gracefully stops receiving data. Finally, `main()` starts the WebSocket server, and `asyncio.run(main())` ensures that the server runs indefinitely.

The `asyncio` library is used to handle asynchronous tasks, ensuring that the server and client can run efficiently without blocking execution. The `websockets` library is used for real-time WebSocket communication, allowing the sender to transfer data and the receiver to accept it seamlessly.

Sender Code(Client):

```
import asyncio
import websockets
async def send_audio():
    uri = "ws://192.168.183.180:8765" # Replace with actual IP
    async with websockets.connect(uri) as websocket:
        print("Connected to server. Sending audio file...")
        # Use the full path of the audio file
        audio_path = "/home/sharon/Desktop/webrtc/audio.mp3" # Change this to your actual path
        with open(audio_path, "rb") as audio_file:
            while chunk := audio_file.read(1024): # Read in 1KB chunks
                await websocket.send(chunk)
        print("Audio file sent successfully!")
asyncio.run(send_audio())
```

The sender code is a WebSocket client that connects to the server using `websockets.connect()`. It reads an audio file (audio.mp3) from the specified path (/home/sharon/Desktop/webrtc/audio.mp3) in **binary read mode** (rb). The file is read in **chunks of 1024 bytes (1KB)** using `while chunk := audio_file.read(1024):`, ensuring efficient data transmission. Each chunk is sent over the WebSocket connection using `await websocket.send(chunk)`. Once the entire file is sent, the connection automatically closes. The function `send_audio()` is run asynchronously using `asyncio.run(send_audio())`, making the process non-blocking.



```
PS D:\webrtc> python server.py
Connection established. Receiving audio file...
Audio file received successfully!
```

MongoDB

On 6th March, a task was given to learn about MongoDB and implement the database and all backend data handling on MongoDB.

- MongoDB was installed in my lap but there was some compatibility issue on the Windows operating system
- Installed in My Linux system, completed the tutorial and learnt about the basics about handling data in MongoDB.

Skills:

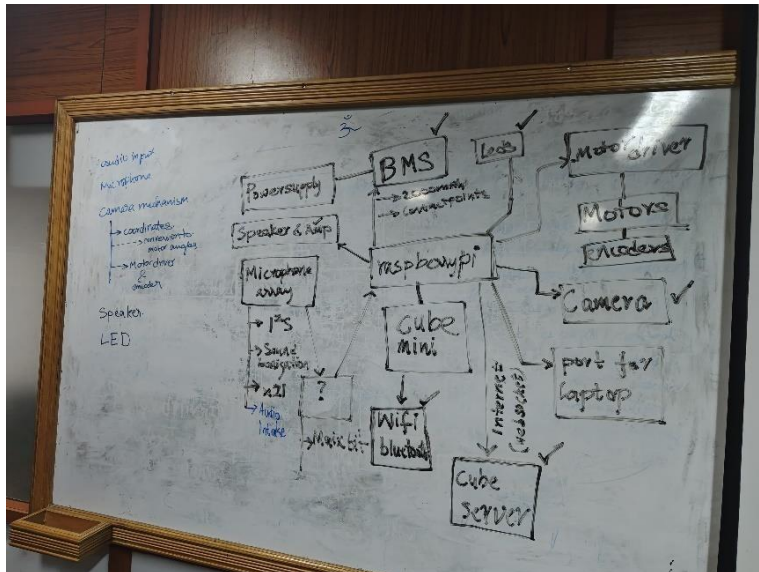
- Started learning about I2S protocol.
- Started learning about Websockets.
- Started learning about MongoDB.

Vacation Task

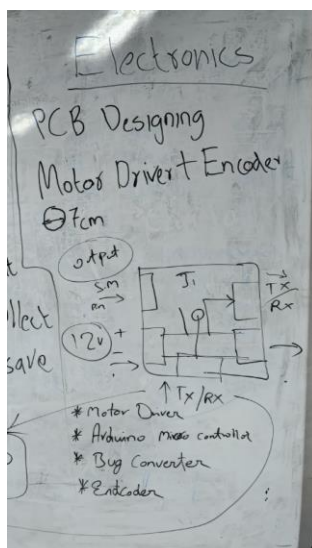
On 28th March we had a discussion about vacation tasks and skills that should be completed within the vacation period (3rd April – 22nd April)

We were all made to explain what all were the updates about their respective domain.

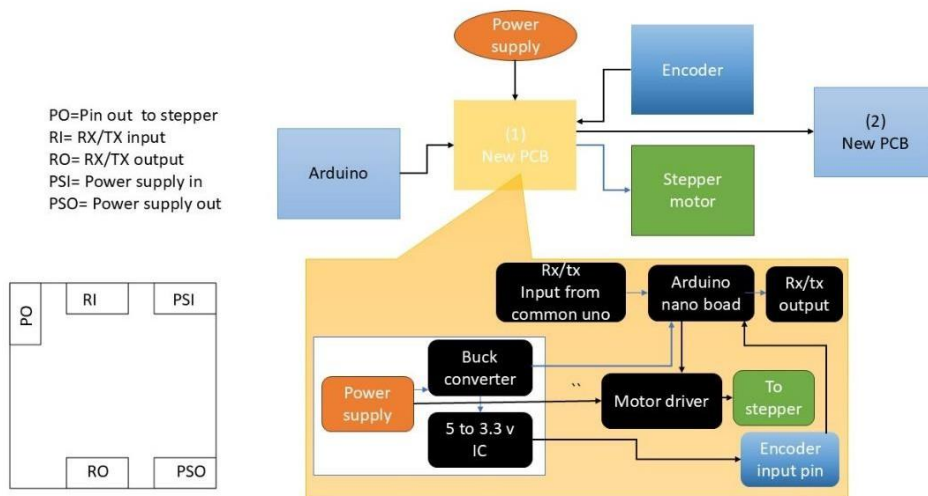
- Each domain explained their respective works and updates
- In our domain we explained a small flow chart of the hardware system.



- Individual tasks were assigned for each people, mine was to design the pcb for the robotic arm which contains microcontroller, motor driver, encoder etc.



Flow chart for the PCB



This was the suggested flow chart for the system.

The major components used were Atmega328p-U Microcontroller, A4988 Motor Driver, AS5600 12-bit Encoder, LM2596 Buck Converter.

The schematics was done in the Altium Designer. All component used in the schematics were selected from the website Digikey.in and LCSC Electronics

DigiKey

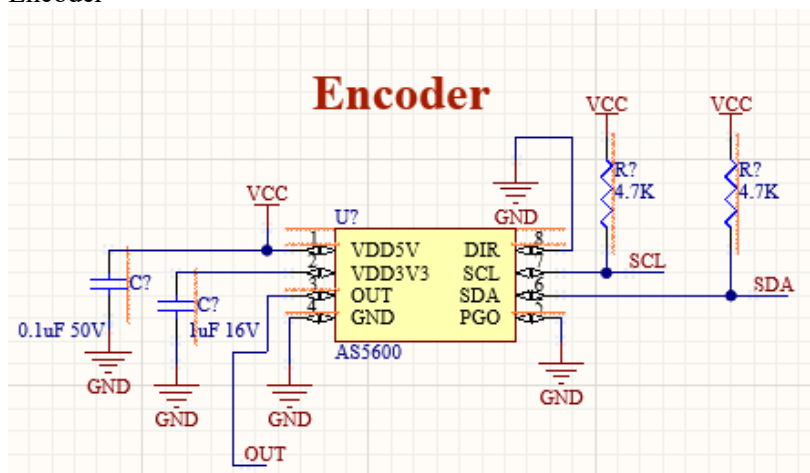


All components used were in stock and available in both these websites. The respective component part numbers for both websites are added in the parameters of each component.

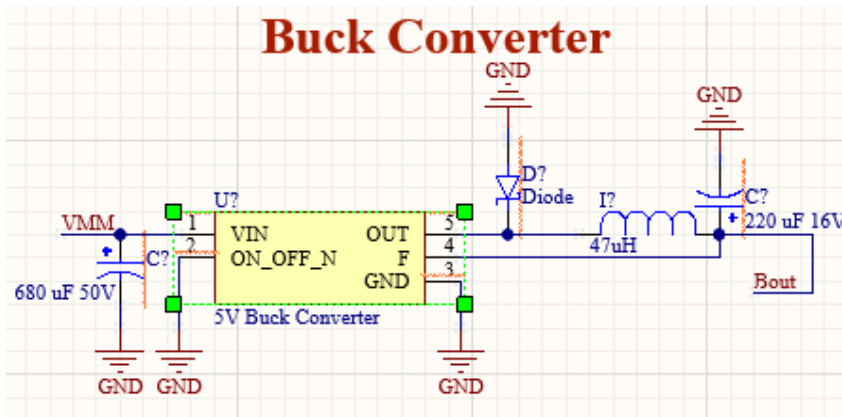
All the missing symbols, footprints and 3D models are filled and was taken from digikey website and from Snapmagic website.

Schematic Breakdown

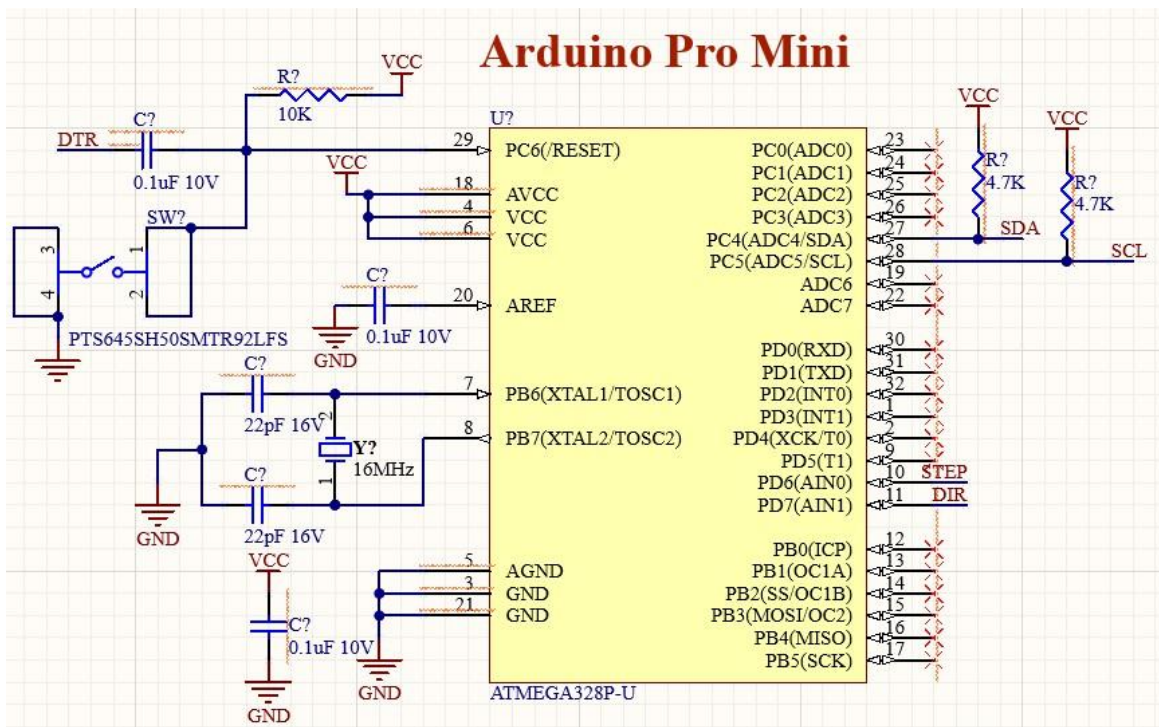
- Encoder



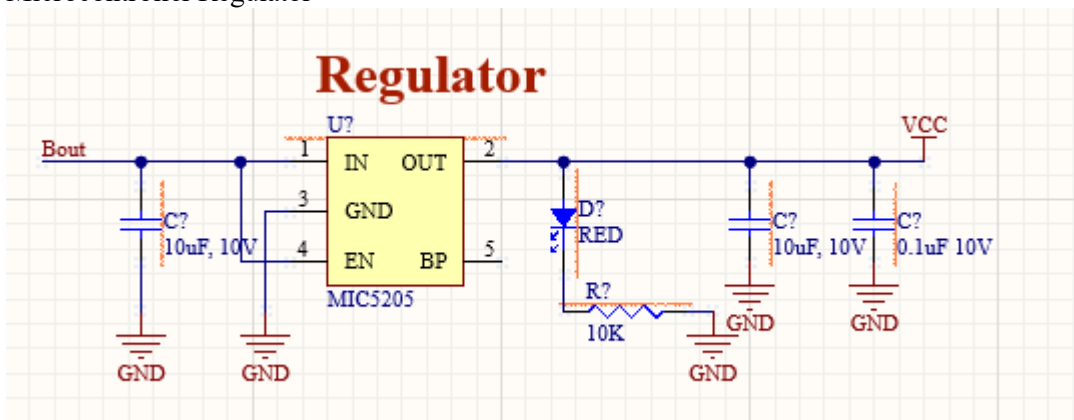
- Buck Convertor



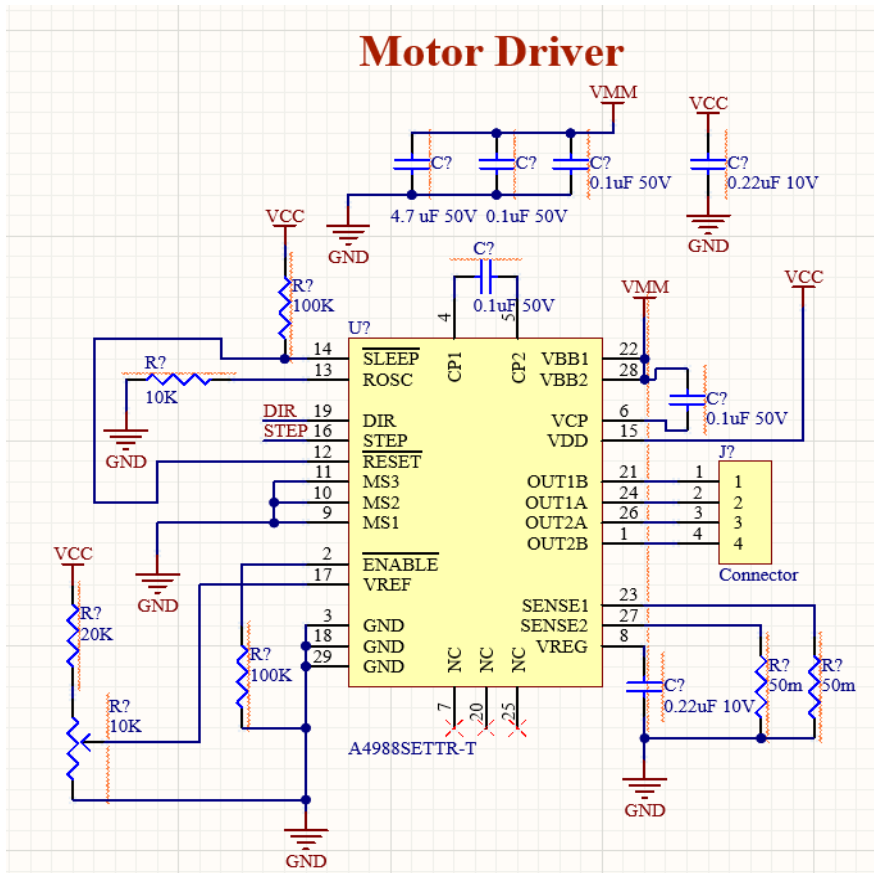
- Microcontroller



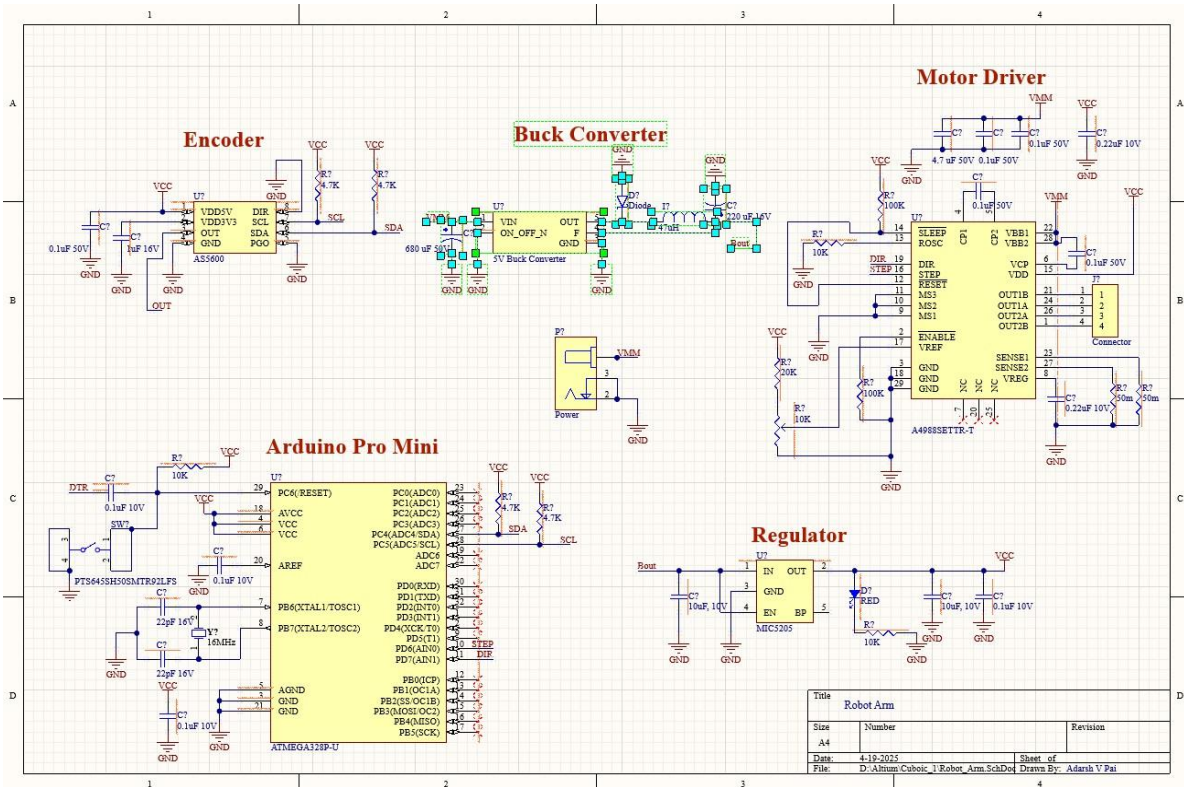
- Microcontroller Regulator



- Motor Driver



Full Schematics



Automatic Delivery Robot

From July 2025 we started working on the electronic and mechanic integration for the automatic delivery robot with 6 axis robotic-arm for a span of 5 months. Engaged in so much trial and error techniques to finalize the motor, driver, encoder, microcontrollers etc.

Design and Development of a 6-Axis Robotic Arm

1. Overview

The 6-axis robotic arm is designed as a functional subsystem of an autonomous delivery robot. Its primary purpose is to perform manipulation tasks such as picking and placing objects, delivering items to specific locations, and interacting with elevator control panels. The arm is engineered to provide precise, repeatable, and reliable motion while operating in confined and real-world environments.

The system integrates mechanical strength with accurate electrical control to ensure smooth operation during repetitive tasks.

2. Mechanical Design

The robotic arm consists of multiple articulated joints connected through rigid links, shafts, and precision bearings to enable smooth and stable motion. A total of six stepper motors are used to drive the arm:

- Two NEMA 23 stepper motors are used at the shoulder and elbow joints, where higher torque is required for lifting and load handling.
- Four NEMA 17 stepper motors are used at the base, wrist, and gripper joints to achieve precise positioning and lightweight motion.

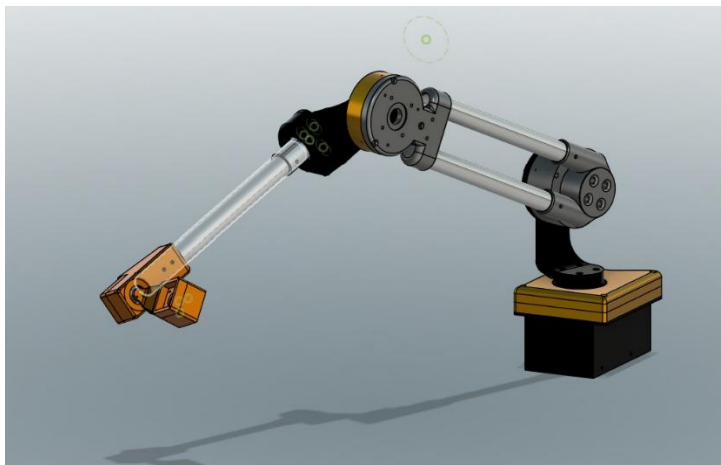
Planetary gear mechanisms are incorporated at high-load joints to increase torque output without sacrificing positional accuracy. Direct-drive configurations are used at lighter joints to achieve faster response and better control. This design provides five effective degrees of freedom, allowing the arm to perform complex manipulation tasks with flexibility and accuracy.

3. Electrical and Control System

The electrical system is designed to provide independent and accurate control of each joint. The four NEMA 17 motors are driven using a CNC motor driver expansion shield, which allows simultaneous control of multiple axes. The two NEMA 23 motors are operated using dedicated high-current motor drivers suitable for higher torque requirements.

Each motor shaft is equipped with an AS5600 magnetic rotary encoder to obtain precise angular position feedback. Since multiple encoders share the same communication protocol, a TCA9548A I²C multiplexer is used to avoid address conflicts on the I²C bus.

An Arduino microcontroller serves as the local controller for the robotic arm. It handles encoder data acquisition, motor control signals, and real-time position monitoring. High-level commands are received from the main robot processor through USB serial communication, ensuring coordinated operation between the robotic arm and the overall delivery system.



4. Material Selection

Material selection was carried out with emphasis on strength, weight reduction, manufacturability, and cost efficiency.

- PLA (Polylactic Acid) is used for 3D printing most structural and housing components, including motor mounts and joint enclosures. PLA offers good dimensional accuracy, low weight, and ease of fabrication, making it suitable for customized robotic parts.
- Stainless steel is used for load-bearing elements such as shafts and rods that experience high mechanical stress. It provides high strength, durability, and resistance to wear, ensuring long-term reliability.

The combination of PLA and stainless steel results in a lightweight yet robust structure that minimizes motor load while maintaining mechanical stability.



Restaurant Robot

From December 2025 to January 2026 we were engaged in developing a restaurant delivery robot running with Raspberry Pi and Arduino Mega but extended the project to upgrade the system that runs the robot

